AI-Driven Traffic Violation Detection and Adaptive Signal Control for Indian Roads

Sachin Nair
Dept. of Computer Engineering
Pillai College of Engineering
Navi Mumbai, India
nsachin23comp@student.mes.ac.in

Aditya Vijay Kumar
Dept. of Computer Engineering
Pillai College of Engineering
Navi Mumbai, India
[email]

Ranjit Iyer
Dept. of Computer Engineering
Pillai College of Engineering
Navi Mumbai, India
[email]

Dr. Siddharth Hariharan
Dept. of Computer Engineering
Pillai College of Engineering
Navi Mumbai, India
[email]

***Abstract*—Indian urban junctions combine heterogeneous, lane-less traffic with frequent helmet, triple-riding and red-light violations, while enforcement and signal timing still depend largely on manual observation and fixed plans. This paper surveys the methods that underpin automated traffic enforcement and adaptive signal control, and presents an integrated system built on them for Indian roads. Video from junction cameras is analysed with a YOLO detector trained on the India Driving Dataset and ByteTrack tracking; dedicated modules check helmet use, triple riding, red-light jumping and speed, read Indian number plates and recognise emergency vehicles for exemption. Every module is precision-first: a violation is reported only when the evidence is certain, uncertain cases are routed to a human reviewer, and confirmed cases become e-challans with fingerprinted evidence on a web platform. For signal control, a Unity-based simulator of an Indian four-arm junction models mixed traffic, red-light jumping, free left turns and bus-stop spillback, and is used to evaluate PRAVAH, a person-weighted, red-patience-aware adaptive controller proposed in this work. In simulation PRAVAH gave the lowest mean wait per person in all six test scenarios, 17–47 % below a queue-pressure adaptive controller in five of them and on par near capacity. The detection modules make few wrong calls on held-out videos at the cost of coverage, which remains the main limitation.**

Keywords*—*Intelligent Transportation Systems, Traffic Violation Detection, Adaptive Traffic Signal Control, Automatic Number Plate Recognition, Helmet Detection, Mixed Traffic, Microscopic Traffic Simulation, Computer Vision

I. INTRODUCTION

Road traffic in Indian cities is unlike the lane-disciplined traffic for which most enforcement and signal-control systems were designed. Two-wheelers make up a large share of vehicles and move between lanes, autorickshaws, buses and trucks share the same carriageway, and left turns are often free. Common violations such as riding without a helmet, carrying three people on a two-wheeler and jumping a red light are frequent, yet they are still largely detected by police officers watching camera feeds or standing at junctions. At the same time, many junctions run fixed signal plans that do not respond to the traffic actually waiting.

Computer vision has made it practical to detect and track vehicles in camera video in real time [6], [7], and datasets collected on Indian roads [8] make it possible to train detectors for local vehicle types. However, an enforcement system has a stricter requirement than a detector benchmark: a wrong challan affects a citizen, so a system must know when it is not sure. Similarly, adaptive signal control has a long history [1]–[4], but most methods measure demand in vehicles and assume orderly queues, which fits Indian mixed traffic poorly.

This paper reviews ten representative works on signal timing, traffic simulation, detection, tracking, number-plate recognition and helmet detection, identifies the gaps that matter for Indian junctions, and describes an integrated system that addresses them. The main contributions are:

1) An end-to-end, precision-first violation pipeline for Indian traffic video: shared detection and tracking, modules for helmet, triple riding, red light and speed, an Indian number-plate reader with explicit READ / NOT SURE / UNREADABLE outputs, and emergency-vehicle exemption, feeding a review and e-challan web platform.
2) A Unity-based microscopic simulator of an Indian four-arm junction with lane-less two-wheelers, red-light jumping, free left turns, bus-stop spillback and stray cattle, with a Python bridge so that controllers written in Python can run the simulated signal.
3) PRAVAH, an adaptive signal controller that weights waiting people rather than vehicles, penalises long red waits that lead to red-light jumping, learns each approach's discharge rate online, avoids feeding a blocked junction and times ambulance priority.
4) An evaluation against fixed-time, Webster-designed and queue-pressure adaptive control on identical simulated traffic, together with held-out video results for the detection modules.

II. LITERATURE SURVEY

A. Signal timing and adaptive control

Webster [1] derived the classical fixed-time design: the optimum cycle length $C_0 = (1.5L + 5)/(1 - Y)$, where L is the lost time per cycle and Y the sum of critical flow ratios, with green time split in proportion to the flow ratios. It remains the basis of signal design guidelines, including the Indian Roads Congress guidance for at-grade intersections, but it assumes demand known in advance. SCOOT [2] made timing traffic-responsive by estimating cyclic flow profiles from upstream detectors and making frequent small changes to splits, offsets and cycle times. Varaiya [3] proposed max-pressure control, which serves the phase with the largest weighted difference between upstream and downstream queues; it needs only local measurements and is proven to stabilise any demand that can

be stabilised. Wei et al. [4] showed that deep reinforcement learning can learn signal policies from data, outperforming fixed-time control on real traffic data, but such policies require extensive training and are hard to explain to a traffic authority.

B. Microscopic traffic simulation

Evaluating signal controllers on a real junction is slow and risky, so controllers are usually compared in simulation. The Intelligent Driver Model of Treiber et al. [5] describes each vehicle's acceleration from its desired speed, safe time headway, minimum gap and comfortable braking, and reproduces the congested traffic states observed on real roads. It is widely used for car following, but it assumes vehicles keep to lanes, so Indian traffic needs additional behaviour such as two-wheelers filtering past queues.

C. Detection and tracking

YOLO [6] framed object detection as a single regression from image to boxes and class probabilities, making real-time detection practical; its later versions are the standard choice for traffic cameras. ByteTrack [7] improved multi-object tracking by associating every detection box, including low-confidence ones, which reduces broken tracks when vehicles are partly hidden, a frequent situation in dense traffic. Varma et al. [8] introduced the India Driving Dataset (IDD), collected in Hyderabad and Bengaluru with classes such as autorickshaw and a label hierarchy suited to unstructured roads, and showed that models trained on European data transfer poorly to Indian roads.

D. Number plates and helmets

Silva and Jung [9] addressed number-plate recognition in unconstrained scenes, where plates are oblique and small, by detecting and rectifying the plate before reading it. Indian plates add further variety in fonts, two-line plates and hand-painted plates. Siebert and Lin [10] built a large annotated video dataset of motorcycles and riders from public roads and trained a deep detector that records helmet use for each rider position, showing that helmet-use observation can be automated from roadside video.

TABLE I
SUMMARY OF THE SURVEYED LITERATURE

Sr.	Author & Year	Methodology / Technique	Key Findings / Relevance to this work
1	Webster (1958) [1]	Fixed-time signal design; delay model	Optimum cycle $C_0 = (1.5L+5)/(1-Y)$ and flow-proportional greens; basis of design guidelines; needs known, steady demand
2	Hunt et al. (1981) [2]	SCOOT: detector-based cyclic flow profiles, incremental optimisation	Traffic-responsive timing of splits, offsets and cycle; requires dense loop detectors and calibration
3	Varaiya (2013) [3]	Max-pressure control from upstream and downstream queues	Decentralised, needs only local queues; provably maximises stable throughput; counts vehicles, not people
4	Wei et al. (2018) [4]	IntelliLight: deep reinforcement learning for signal control	Learns policies that beat fixed-time control on real data; costly training and limited explainability
5	Treiber et al. (2000) [5]	Intelligent Driver Model for car following	Reproduces empirical congested states with few parameters; assumes lane-based traffic
6	Redmon et al. (2016) [6]	YOLO single-stage object detection	Real-time detection as one regression; foundation of modern traffic-camera detectors
7	Zhang et al. (2022) [7]	ByteTrack: associate every detection box	Fewer missed and broken tracks under occlusion; suits dense traffic
8	Varma et al. (2019) [8]	IDD: India Driving Dataset for unstructured roads	Indian classes incl. autorickshaw; shows poor transfer of models trained on European data
9	Silva and Jung (2018) [9]	Plate detection with rectification in unconstrained scenarios	Handles oblique, small plates before OCR; plate styles differ from Indian plates
10	Siebert and Lin (2020) [10]	Deep-learning detection of motorcycle helmet use per rider position	Large annotated road-video dataset; shows helmet observation can be automated

E. Gaps for Indian junctions

Table I summarises the surveyed works. Four gaps stand out. First, signal controllers measure demand in vehicles, while Indian junctions mix two-wheelers carrying one or two people with buses carrying forty, so a vehicle count is a poor measure of who is waiting. Second, long red waits cause red-light jumping in practice, but controllers do not treat waiting time as a safety risk. Third, vision benchmarks report accuracy, while enforcement needs a system that abstains when unsure. Fourth, enforcement and signal control are usually studied separately, although they use the same cameras and the same vehicle tracks.

III. PROPOSED SYSTEM

The proposed system (Fig. 1) has one shared video-analysis layer feeding two uses: enforcement, which produces evidence and e-challans, and signal control, which uses the measured traffic to choose signal timing. All modules follow one rule: an automatic decision is made only when the evidence is certain; otherwise the case goes to a human reviewer, and nothing is printed or charged on a guess.

A. Shared detection and tracking

Each video is sampled at five frames per second. A YOLO11 detector trained on the full IDD [8] finds vehicles, riders and people, and ByteTrack [7] keeps one identity per vehicle across frames; track fragments of the same vehicle are merged when one starts where another ended. The same tracks are reused by every module, so a vehicle is decided once even if several modules examine it.

B. Violation modules

The helmet module classifies each rider's head on enlarged crops and votes over at least three frames, reporting "no helmet" only above a strict threshold. The triple-riding module counts riders on each two-wheeler with a "not sure" band between its two thresholds. The red-light module reads the signal state and checks stop-line crossing during red. The speed module reports a calibrated speed range with a confidence level rather than a single number. Lane structure is estimated from traffic motion for context only and never creates a challan.

C. Identity and exemption

The number-plate reader searches inside each tracked vehicle, enlarged to 640 pixels, reads each plate crop with a recogniser fine-tuned on Indian plates, votes character by character across frames and checks the Indian registration format. Each vehicle is labelled READ, NOT SURE or UNREADABLE, and only READ plates are printed. An emergency-vehicle classifier on CLIP image features exempts ambulances, police and fire vehicles, sending borderline cases to review.

D. Review and e-challan platform

A FastAPI and React web platform stores each analysis, shows the video with every finding on a timeline, routes uncertain findings to reviewers and issues a PDF e-challan for confirmed cases. Every challan carries fingerprinted evidence frames and a plain-language explanation of why it was raised.

E. Signal control and simulation

The signal-control part is evaluated in a simulator of an Indian four-arm junction, described in Section IV. The measured traffic of an analysed video can be loaded into the simulator, so that controllers can be compared on traffic recorded at a real junction; such results are always labelled as simulated.

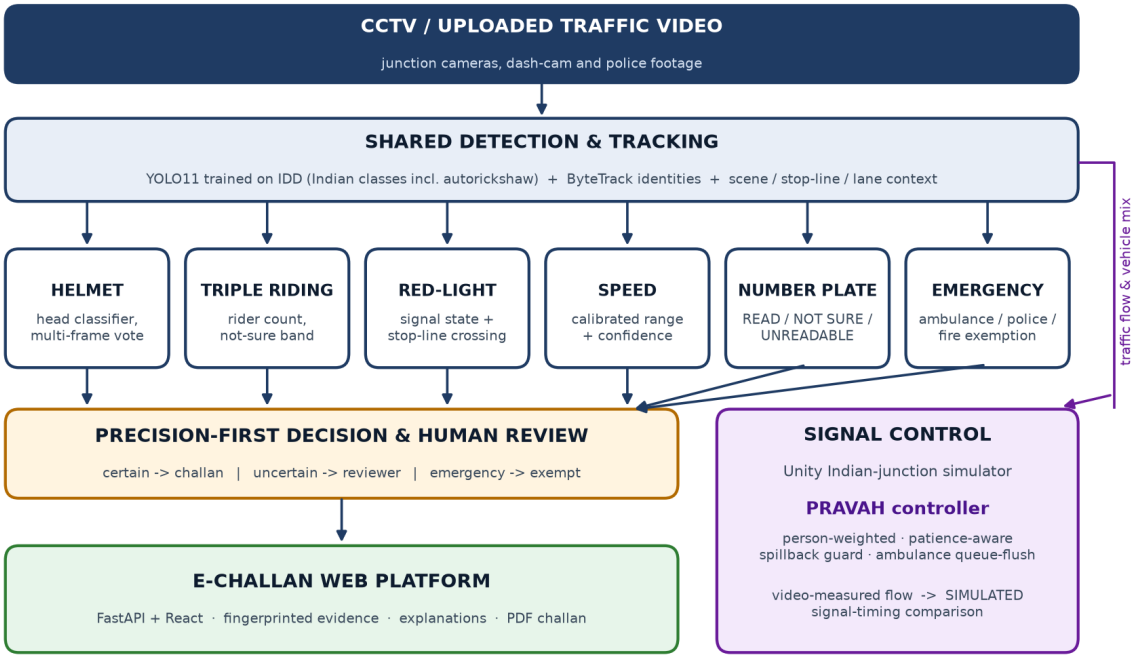


Fig. 1. Architecture of the proposed system: shared video analysis feeding precision-first enforcement and signal control.

F. PRAVAH controller

PRAVAH (Person-weighted, Red-patience-Aware, Vehicle-mix Adaptive Headway control) serves one approach at a time, which is common at Indian junctions. It uses only what the junction camera reports within 100 m of the stop line. A person who has waited w seconds contributes an urgency

$$U(w) = 1 + (w / \tau)^2, \quad \tau = 75 \text{ s} \tag{1}$$

so that long waits, which lead to red-light jumping, become expensive quickly. Each waiting vehicle counts its typical occupants (about 40 for a bus, 1.4 for a two-wheeler). The time needed to clear approach a is

$$C_a = 2 + Q_a / \hat{s}_a \tag{2}$$

where Q_a is the queued load in passenger-car units and $\hat{s}_a$ the discharge rate, initialised from the Indian Roads Congress saturation flow of 525 PCU/h per metre of approach width and learned online from the vehicles that actually cross during green. The gain of serving approach a next is

$$G_a = \Sigma\, p_i\, U(w_i) \cdot \varphi(o_a) / (C_a + L) \tag{3}$$

where p_i are occupants, L = 5 s is the amber and all-red loss, and $\varphi(o) = \text{clip}(1 - (o - 0.5)/0.4, 0, 1)$ discounts an approach whose exit is more than half full, so the controller does not feed a blocked junction box. After clearing its planned queue, a green continues only while it relieves more urgency per second ($\hat{s}_a$ times the urgency per PCU of the remaining queue) than the best other approach would; any approach that has waited 90 s is served next. For an ambulance, the switch is timed so that its queue has cleared when it arrives: the green changes when its estimated arrival time falls below its clearance time plus L and 2 s.

IV. IMPLEMENTATION

The video pipeline is written in Python with Ultralytics YOLO11, ByteTrack, OpenCV and PyTorch; detectors and classifiers were trained on Kaggle T4 GPUs. The number-plate recogniser is a fine-tuned compact transformer OCR model exported to ONNX. The web platform uses FastAPI with SQLite and a React and TypeScript front end.

The simulator is built in Unity 6 with C#. Its core is a headless microscopic model with a 0.1 s step: vehicles follow the Intelligent Driver Model [5], drive on the left, and arrive with a typical Indian mix (38 % two-wheelers, 28 % cars, 16 % autorickshaws, 8 % trucks, 6 % buses, 4 % bicycles), weighted with Indian Roads Congress passenger-car units (0.5 for a two-wheeler, 1.2 for an autorickshaw, 2.2 for a bus or truck). Two-wheelers take lateral positions across the lane and creep past the stop line; each driver has a patience after which they jump the red; left turns are free; buses and autos stop just past the junction; a vehicle stuck in the junction box blocks cross traffic; stray cattle occasionally block the kerb lane. A three-dimensional view with day, night and monsoon lighting and eight camera modes, including junction CCTV views with detection boxes, is used for demonstration (Fig. 2).

Four controllers are compared on identical arrivals: a fixed 30 s rotation, a fixed-time plan designed with Webster's method [1] from the true flows, a queue-pressure adaptive controller, and PRAVAH. A TCP bridge lets Python controllers run the simulated signal; a shadow test, in which the C# and Python implementations of PRAVAH received identical states, found no disagreement in 83,702 decisions. The web platform can run all controllers on the traffic measured from an analysed video.



Fig. 2. The Unity junction simulator: chase view behind an ambulance on an Indian arterial (CC0 photographic sky and road textures).

V. RESULTS AND DISCUSSION

A. Signal control in simulation

Each controller was run for 30 simulated minutes in six scenarios with three random seeds, all controllers receiving the same arrivals in each seed. Table II and Fig. 3 report the mean wait per person, that is, time stopped before the stop line weighted by occupants. PRAVAH gave the lowest wait per person in every scenario. Compared with the queue-pressure adaptive controller it was 42 % lower in balanced city traffic, 17 % lower in the office rush, 35 % lower on the market road, 47 % lower at night and 21 % lower in the ambulance-heavy scenario. Ambulances waited 0.1–0.4 s on average in five scenarios and 7.6 s near capacity.

The gains are not uniform. Near capacity PRAVAH was on par with the adaptive controller (73.8 s against 75.7 s), with a longer worst-case wait (295 s against 268 s) and slightly more red-light jumps. In the ambulance-heavy scenario its worst wait was 148 s against 101 s. These results come from a simulator whose driver-behaviour parameters (patience, dwell times) are estimates, so they show how the controllers compare rather than predict real delays.

Running an earlier, rule-based adaptive controller of the project through the Python bridge gave a further finding: it performed well in moderate traffic but broke down near capacity, with a mean wait of 98.7 s, throughput falling to 2,339 vehicles/h and 486 vehicles left queued. The lack of a waiting-time limit and of a spillback check explains this, and motivated both features in PRAVAH.

TABLE II
MEAN WAIT PER PERSON (S) IN SIMULATION — 30 MIN, 3 SEEDS, IDENTICAL ARRIVALS

Scenario	Fixed 30 s	Webster	Queue-pressure adaptive	PRAVAH (proposed)	PRAVAH vs adaptive
City balanced	38.9	36.3	26.9	**15.5**	−42 %
Office rush (N–S)	89.8	36.3	23.3	**19.4**	−17 %
Market road (cattle, bus stops)	41.5	30.7	22.3	**14.4**	−35 %
Peak near capacity	87.0	98.1	75.7	**73.8**	−3 %
Night	31.6	11.1	5.5	**2.9**	−47 %
Ambulance corridor	41.0	29.6	20.2	**15.9**	−21 %

TABLE III
DETECTION MODULES ON HELD-OUT DATA

Module	Held-out test	Result	Design choice
Number-plate reader	299 Indian plate images	81.6 % plates and 95.6 % characters exact; at the operating point it answers for 64 %, 99.5 % correct	Reads only when certain
Helmet	7 unseen videos, 367 riders	4 wrong decisions; 44 % sent to review; 27 of 81 no-helmet riders caught	Avoid wrong challans
Triple riding	Held-out cameras, 980 bikes, 125 triples	50 triples caught (40 %), 0 wrong, 8.3 % not sure	Never wrong on test
Emergency exemption	170 emergency, 2,450 ordinary vehicles	58 % exempted automatically, 84 % exempted or reviewed; 0 ordinary vehicles wrongly exempted	No wrongful exemption
Speed	Calibrated demonstration video	Mean error 9.1 %; 90 % range contains the true speed for 90 % of vehicles	Range, not a point value

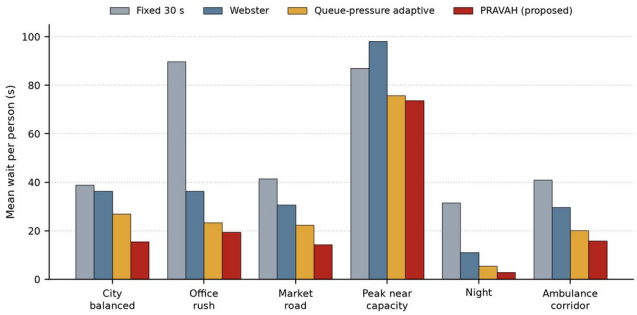


Fig. 3. Mean wait per person by controller and scenario (lower is better).

B. Detection modules on held-out data

Table III summarises the detection modules on data not used for training. All operating points were chosen to avoid wrong decisions, so coverage is the limiting factor. On still plate images the reader answers for 64 % of plates and is correct for 99.5 % of those; on street video, however, plates are often too small, and only about 1–3 % of vehicles received a READ result in the test clips. Retraining the plate detector on enlarged vehicle crops from Indian street video is in progress to address this.

VI. FUTURE SCOPE

The most important next step is coverage of the detection modules on street-level video, where plates and riders are small: retraining the plate detector on enlarged vehicle crops, and labelling more camera views, including phone footage, for helmet and triple-riding detection. For signal control, a sensitivity study over driver patience, dwell times, camera range and demand is under way to test whether PRAVAH's advantage holds when the simulator's assumptions change. Field validation with measured queue lengths at a real junction, coordination of neighbouring junctions along a corridor, pedestrian phases and deployment on edge devices beside the camera are further directions.

VII. CONCLUSION

This paper reviewed ten works spanning signal timing, traffic simulation, detection, tracking, number-plate recognition and helmet detection, and identified four gaps for Indian junctions: demand measured in vehicles rather than people, waiting time not treated as a cause of red-light jumping, detection evaluated by accuracy rather than by its ability to abstain, and enforcement separated from signal control. The proposed system addresses them with a precision-first violation pipeline and e-challan platform, an Indian junction simulator, and the PRAVAH controller. In simulation PRAVAH reduced the mean wait per person in every scenario tested, by 17–47 % against a queue-pressure adaptive controller in five of six scenarios, while being on par near capacity. On held-out video the detection modules made few wrong decisions, and their limited coverage on street-level footage is the main area for further work.

REFERENCES

[1] F. V. Webster, "Traffic signal settings," Road Research Technical Paper No. 39, Road Research Laboratory, HMSO, London, 1958.

[2] P. B. Hunt, D. I. Robertson, R. D. Bretherton and R. I. Winton, "SCOOT – a traffic responsive method of coordinating signals," TRRL Laboratory Report 1014, Transport and Road Research Laboratory, Crowthorne, 1981.

[3] P. Varaiya, "Max pressure control of a network of signalized intersections," Transportation Research Part C: Emerging Technologies, vol. 36, pp. 177–195, 2013.

[4] H. Wei, G. Zheng, H. Yao and Z. Li, "IntelliLight: A reinforcement learning approach for intelligent traffic light control," in Proc. 24th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining, 2018, pp. 2496–2505.

[5] M. Treiber, A. Hennecke and D. Helbing, "Congested traffic states in empirical observations and microscopic simulations," Physical Review E, vol. 62, no. 2, pp. 1805–1824, 2000.

[6] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788.

[7] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in Proc. European Conf. Computer Vision (ECCV), 2022, pp. 1–21.

[8] G. Varma, A. Subramanian, A. Namboodiri, M. Chandraker and C. V. Jawahar, "IDD: A dataset for exploring problems of autonomous navigation in unconstrained environments," in Proc. IEEE Winter Conf. Applications of Computer Vision (WACV), 2019, pp. 1743–1751.

[9] S. M. Silva and C. R. Jung, "License plate detection and recognition in unconstrained scenarios," in Proc. European Conf. Computer Vision (ECCV), 2018, pp. 580–596.

[10] F. W. Siebert and H. Lin, "Detecting motorcycle helmet use with deep learning," Accident Analysis & Prevention, vol. 134, art. 105319, 2020.